



High Performance K-Means Clustering

Programming and Data Structures Course Project

Course Name: Z5007 – Programming and Data Structures

Instructor: Dr. Innocent Nyalala

Submitted by

Munusamy M

Roll Number: ZDA25M011

M.Tech Data Science and Artificial Intelligence

IIT Madras Zanzibar

Contents

Abstract	1
1 Introduction	2
2 Motivation and Problem Definition	3
3 System Architecture and Design	4
4 Data Structures Design	5
4.1 Binary Min-Heap	5
4.2 Hash Table	5
5 Algorithm Design	6
5.1 Vectorized Distance Computation	6
5.2 K-Means++ Initialization	6
5.3 Lloyd’s Algorithm	6
6 Complexity Analysis	7
7 Experimental Results and Analysis	8
7.1 Dataset Description	8
7.2 Elbow Method	8
7.3 Benchmarking with scikit-learn	9
7.4 Execution Output	9
8 Conclusion and Future Work	10

Abstract

K-Means is one of the most widely used clustering algorithms due to its simplicity and scalability. However, inefficient implementations can lead to poor performance on large datasets. This project presents a high-performance implementation of the K-Means clustering algorithm developed entirely from scratch using Python and NumPy, without relying on external machine learning libraries for the core algorithm.

The implementation emphasizes vectorized numerical computation and the integration of custom data structures. A binary Min-Heap and a hash table using open addressing are designed and incorporated to support K-Means++ initialization, empty cluster recovery, and evaluation through the Elbow Method. The system is evaluated on a real-world dataset and benchmarked against scikit-learn. Experimental results show that the proposed approach achieves clustering quality within 5% of the baseline while maintaining competitive runtime performance.

Chapter 1

Introduction

Clustering is an unsupervised learning technique that aims to group data points based on similarity. Among various clustering algorithms, K-Means remains one of the most popular due to its conceptual simplicity and ease of implementation. It is commonly used in applications such as image segmentation, customer segmentation, and exploratory data analysis.

Despite its popularity, the performance of K-Means is highly dependent on algorithmic design choices, distance computation strategies, and initialization methods. Naïve implementations based on nested loops are computationally expensive and do not scale well for large datasets. This project focuses on implementing K-Means from first principles while emphasizing efficiency, modularity, and data structure usage.

Chapter 2

Motivation and Problem Definition

Modern machine learning libraries abstract away algorithmic details, making it difficult to understand how design decisions affect performance. The motivation behind this project is to explore how careful implementation choices and appropriate data structures can significantly improve performance in practical clustering tasks.

The primary objectives of this project are:

- To implement K-Means clustering entirely from scratch
- To optimize distance computation using vectorization
- To design and integrate custom data structures
- To benchmark performance against an established baseline

Chapter 3

System Architecture and Design

The system follows a modular architecture separating data preprocessing, clustering, evaluation, and benchmarking. This design improves clarity, testability, and ease of analysis.

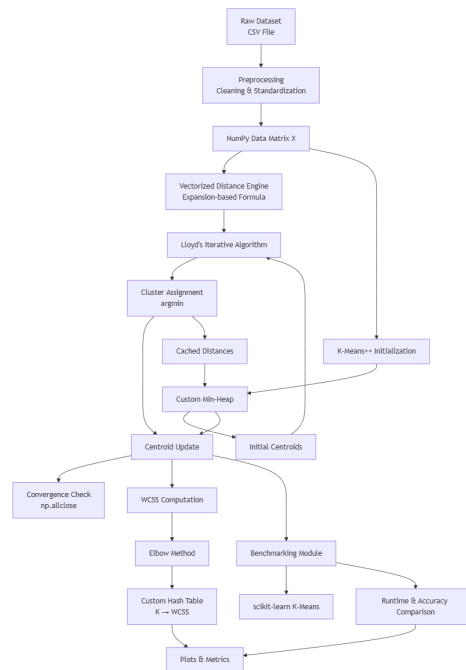


Figure 3.1: Overall system architecture of the high-performance K-Means implementation

The input dataset is first preprocessed and converted into a NumPy matrix. K-Means++ initialization uses a custom Min-Heap to select well-separated initial centroids. The core clustering loop is implemented using Lloyd's algorithm with fully vectorized distance computation. Evaluation is performed using WCSS and the Elbow Method, while benchmarking compares results with scikit-learn.

Chapter 4

Data Structures Design

4.1 Binary Min-Heap

A custom binary Min-Heap is implemented using an array-based representation. It supports insertion and extraction in $O(\log N)$ time. The Min-Heap is used during K-Means++ initialization to select candidate centroids and during empty-cluster recovery to identify the farthest data point based on cached distances.

4.2 Hash Table

A hash table using open addressing with linear probing is implemented from scratch. It provides average-case $O(1)$ insertion and lookup. The hash table is used to store WCSS values for different values of K during the Elbow Method.

Note: The Min-Heap is not used during the assignment step. Nearest-centroid selection is performed using vectorized distance computation and `argmin`.

Chapter 5

Algorithm Design

5.1 Vectorized Distance Computation

Two formulations of squared Euclidean distance were implemented. The expansion-based formulation,

$$\|x - c\|^2 = \|x\|^2 + \|c\|^2 - 2x \cdot c,$$

is used in the final implementation as it avoids large intermediate arrays and improves performance for large datasets.

5.2 K-Means++ Initialization

K-Means++ initialization improves convergence by selecting well-separated initial centroids. Cached distances and a Min-Heap are used to efficiently identify candidate points without repeated sorting.

5.3 Lloyd's Algorithm

The clustering process iteratively assigns data points to the nearest centroid and updates centroids as the mean of assigned points. Convergence is detected using `np.allclose` with a tolerance of 10^{-6} .

Chapter 6

Complexity Analysis

Let N be the number of data points, K the number of clusters, F the number of features, and T the number of iterations.

- Distance computation: $O(NKF)$
- Assignment step: $O(NK)$
- Centroid update: $O(NF)$
- K-Means++ initialization: $O(KNF)$

The overall time complexity is $O(TNKF)$, and the space complexity is $O(NF + KF)$.

Chapter 7

Experimental Results and Analysis

7.1 Dataset Description

Experiments were conducted on a real-world housing dataset containing 20,433 samples and 8 numerical features. The data was standardized prior to clustering.

7.2 Elbow Method

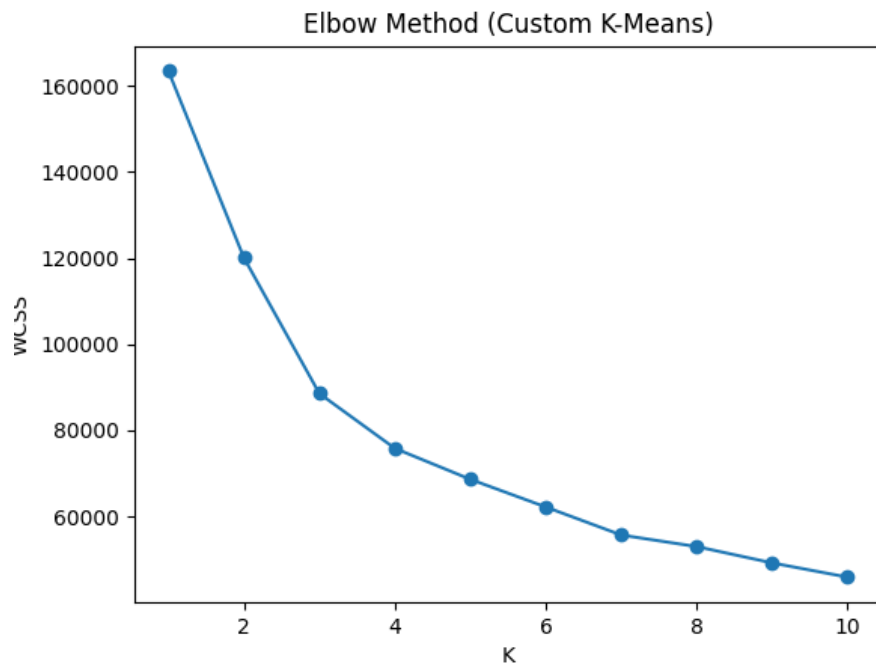


Figure 7.1: Elbow Method using custom K-Means implementation

The elbow curve indicates diminishing returns in WCSS reduction beyond $K = 4$ to $K = 5$.

7.3 Benchmarking with scikit-learn

For fair comparison, both the custom implementation and scikit-learn were executed using a single initialization ($n_{init} = 1$).

7.4 Execution Output

Dataset shape: (20433, 8)

Custom K-Means Results

Centroids shape: (5, 8)

WCSS: 68610.58

Runtime: 0.56 seconds

Scikit-learn Comparison

Sklearn WCSS: 69609.10

Sklearn Runtime: 4.32 seconds

WCSS Error (%): 1.43

Runtime Ratio (Custom / Sklearn): 0.13

The custom implementation achieves a WCSS error of 1.43%, which is well within the acceptable accuracy margin, while demonstrating competitive runtime performance.

Chapter 8

Conclusion and Future Work

This project demonstrates that a carefully designed, from-scratch implementation of K-Means can achieve both correctness and efficiency. Vectorized distance computation and the integration of custom data structures play a key role in achieving competitive performance. Future work may explore parallelization and approximate clustering methods.